

Boolean Logic and Comparisons

Lecture 6

CS 1001

Kirk Duran

Today

- ▶ How Python represents logical truth using the `bool` type.
- ▶ How comparison expressions inspect values and produce `True` or `False`.
- ▶ Why assignment with `=` and equality comparison with `==` have different semantics.
- ▶ How `and`, `or`, and `not` combine or negate Boolean claims.
- ▶ How truth tables, precedence, and parentheses help analyze compound Boolean expressions.

Key idea

This lecture ends with one capability: Python can compute whether a claim is true or false.

Part 1: Boolean Values

Boolean Logic Begins with Claims

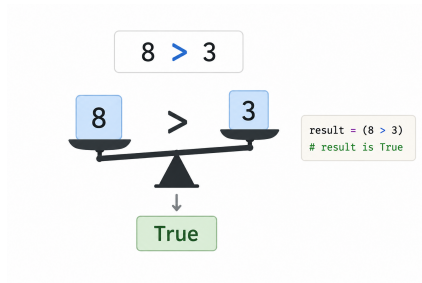
- ▶ Previous lectures established that expressions evaluate to values.
- ▶ Programs also need to represent claims about those values.
- ▶ A logical claim has one of two outcomes: True or False.

Opening expression

$8 > 3$

Example

The expression evaluates to the Boolean value True.



The `bool` Type Represents Two Logical Values

Definition

A **Boolean value** represents one of two logical states: True or False.

Boolean literals

True False

Boolean variables

```
is_raining = True  
is_sunny = False  
has_passed = True
```

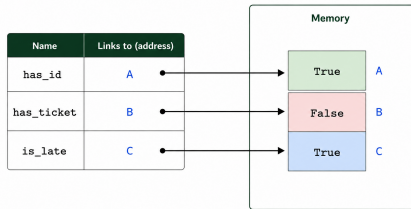
- ▶ Boolean literals are capitalized in Python.
- ▶ Their Python type is `bool`.
- ▶ Boolean values can be stored, printed, assigned, and combined like other values.

Boolean Values Can Be Stored in Variables

Example state

```
has_id = True  
has_ticket = False  
is_late = True
```

- ▶ The variable name refers to a Boolean value.
- ▶ Evaluating the name retrieves its current Boolean value.
- ▶ This is the same variable model introduced in Lecture 5.



Booleans and Integers in Python

- ▶ In Python, `bool` is a subtype of `int`.
- ▶ Numerically, `True` behaves like 1 and `False` behaves like 0.
- ▶ This implementation relationship is useful context, but the primary abstraction is logical truth.

Conceptual binary representation

`True` → 1
`False` → 0

Low-level representation idea

`True` → 00000001
`False` → 00000000

Abstraction boundary

Python objects have implementation details beyond a single raw bit. Use `True/False` as the programming model.

Part 2: Comparison Expressions

From Values to Claims About Values

A stored value

`temperature = 72`

- ▶ The variable `temperature` refers to the value 72.

A claim about the value

`temperature > 70`

- ▶ The expression asks whether the current value is greater than 70.
- ▶ The result is True.

Definition

A **comparison expression** compares values and evaluates to a Boolean value.

Comparison Expressions Produce Boolean Values

Examples

<code>8 > 3</code>	$\longrightarrow$	True
<code>8 == 3</code>	$\longrightarrow$	False
<code>8 != 3</code>	$\longrightarrow$	True

Expression model

`8 > 3`
 $\downarrow$ evaluate
True

Key idea

A comparison is an expression: it computes a value rather than performing a control-flow action.

Python Provides Six Fundamental Comparison Operators

Operator	Meaning	Operator	Meaning
<	less than	<=	less than or equal to
>	greater than	>=	greater than or equal to
==	equal to	!=	not equal to

Examples

5 > 3 → True 5 < 3 → False 5 == 5 → True
5 != 3 → True 5 >= 5 → True 5 <= 3 → False

Comparisons Can Inspect Values Stored in Variables

Example

```
temperature = 72
```

```
temperature > 70
```

1. Retrieve the current value of temperature: 72.
2. Compare `72 > 70`.
3. Produce the Boolean value `True`.

Image Placeholder

comparison-variable-evaluation.png

Part 3: Equality Versus Assignment

Assignment and Equality Comparison Have Different Semantics

Assignment

`x = 5`

- ▶ Evaluates the right-hand side.
- ▶ Associates `x` with the resulting value.
- ▶ Changes program state.

Equality comparison

`x == 5`

- ▶ Compares two values.
- ▶ Produces `True` or `False`.
- ▶ Does not reassign `x`.

Part 4: Boolean Values from Expressions

Boolean Values Can Be Assigned Directly or Computed

Direct Boolean assignment

```
enrolled = True
```

Computed Boolean assignment

```
units = 12  
enrolled = units >= 1
```

Evaluation

```
units >= 1  $\longrightarrow$  12 >= 1  $\longrightarrow$  True
```

Key idea

A comparison expression can produce the Boolean value that assignment then stores.

Boolean Expressions Use the Same Expression Model

Arithmetic expression

`x = 5 + 3`

`5 + 3  $\longrightarrow$  8`

1. Evaluate the expression on the right.
2. Obtain one resulting value.
3. Associate the name on the left with that value.

Boolean expression

`enrolled = units >= 1`

`units >= 1  $\longrightarrow$  True`

Part 5: Boolean Operators

Boolean Operators Combine or Modify Claims

Individual claims

```
age >= 18  
has_ticket
```

Combined claim

```
age >= 18 and has_ticket
```

Definition

A **Boolean operator** combines or modifies Boolean values and produces another Boolean result.

Python Boolean operators

and

or

not

Boolean AND Requires Both Claims to Be True

A	B	A and B
False	False	False
False	True	False
True	False	False
True	True	True

Example

```
has_id = True  
has_ticket = True  
  
has_id and has_ticket  
→ True
```

Key idea

and is true only when both component claims are true.

Boolean OR Requires At Least One True Claim

A	B	A or B
False	False	False
False	True	True
True	False	True
True	True	True

Example

```
is_weekend = False  
is_holiday = True  
  
is_weekend or is_holiday  
→ True
```

Key idea

Python's logical `or` is inclusive: it is also true when both component claims are true.

Boolean NOT Reverses a Truth Value

A	not A
False	True
True	False

Example

```
is_late = True
```

```
not is_late
```

```
→ False
```

- ▶ not is a unary Boolean operator.
- ▶ It negates the truth value of one Boolean operand.

Read Boolean Expressions as Logical Claims

Python	Plain-language claim
<code>has_id and has_ticket</code>	has identification and has a ticket
<code>is_weekend or is_holiday</code>	it is the weekend or a holiday
<code>not is_late</code>	it is not late
<code>temperature >= 70 and is_sunny</code>	at least 70 degrees and sunny

In class

Translate: “The student has completed the prerequisite and is enrolled.”

Part 6: Truth Tables

Truth Tables Enumerate Every Boolean Input Combination

Definition

A **truth table** lists every possible combination of Boolean inputs and the result of a logical expression for each combination.

- ▶ Truth tables provide a systematic method for analyzing Boolean expressions.
- ▶ They clarify the exact semantics of **and**, **or**, and **not**.
- ▶ They are useful for reasoning about and debugging compound logical statements.

Key idea

For two Boolean inputs, there are four possible input combinations.

Combined Truth Table for **and** and **or**

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Key idea

The table encodes the complete behavior of each binary Boolean operator for Boolean operands.

Truth Tables Can Analyze Compound Expressions

Expression

A and not B

A	B	not B	A and not B
True	True	False	False
True	False	True	True
False	True	False	False
False	False	True	False

Use Truth Tables to Reason Systematically

A and B

Ask whether both claims are true.

A or B

Ask whether at least one claim is true.

not A

Reverse the truth value of A.

In class

Without running Python, determine the result of `True and not False` and justify each step.

Part 7: Compound Boolean Expressions

Compound Boolean Expressions Combine Multiple Claims

Definition

A **compound Boolean expression** combines two or more Boolean expressions using Boolean operators and evaluates to one Boolean value.

Example

```
age = 19  
has_ticket = True  
age >= 18 and has_ticket
```

Evaluation

$19 \geq 18 \text{ and True} \longrightarrow \text{True and True} \longrightarrow \text{True}$

Evaluate Compound Expressions in Stages

Example

```
temperature = 72
```

```
is_sunny = True
```

```
temperature >= 70 and is_sunny
```

1. Substitute current variable values.
2. Evaluate comparisons.
3. Evaluate Boolean operators.

Stages

```
72 >= 70 and True
```

↓

```
True and True
```

↓

```
True
```

Example: Assigning the Result of a Compound Expression

Statement

`x = 3 > 4 and 5 < 6`

1. `3 > 4` $\longrightarrow$ `False`
2. `5 < 6` $\longrightarrow$ `True`
3. `False and True` $\longrightarrow$ `False`
4. Assignment associates `x` with `False`.

Key idea

The right-hand side is fully evaluated before its Boolean result is assigned.

Practice: Evaluate Compound Logical Statements

Expressions from the prior lecture deck

`x = 3 > 4 and 5 < 6`

`y = 4 == 4 or 7 != 7`

`z = not (10 <= 15)`

`a = (8 > 2 and 3 < 5) or (6 == 6)`

`b = not (5 > 2) and (3 >= 3)`

`c = 7 != 7 or (2 + 2 == 4 and 9 < 10)`

`d = (3 < 1) and not (5 == 5)`

In class

Evaluate each expression hierarchically: arithmetic → comparisons → Boolean operations → final Boolean value.

Part 8: Operator Precedence and Parentheses

Operator Precedence Determines Evaluation Order

Practical precedence for today's operators

1. Arithmetic
2. Comparisons
3. `not`
4. `and`
5. `or`

Example

`age + 1 >= 18 and has_ticket`

Example

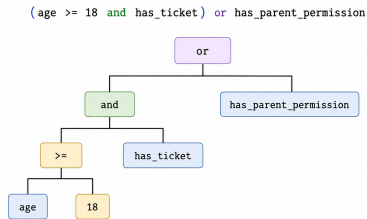
Conceptually: `((age + 1) >= 18) and has_ticket`

Parentheses Make Logical Structure Explicit

Expression

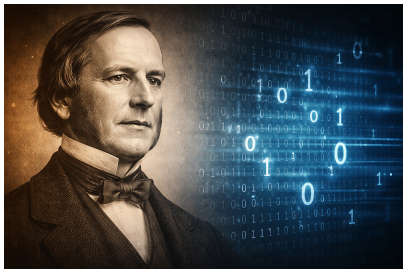
`(age >= 18 and has_ticket) or has_parent_permission`

- ▶ Parentheses can override normal precedence.
- ▶ Even when they do not change the result, they can make intended grouping easier to read.
- ▶ Use parentheses when they clarify the logical structure of a compound claim.



George Boole and the Origins of Boolean Algebra

- ▶ George Boole (1815–1864) was an English mathematician and logician.
- ▶ He developed an algebraic system for reasoning about logical propositions.
- ▶ Boolean algebra became foundational to formal logic, digital circuits, and computing.
- ▶ Claude Shannon later connected Boolean algebra to electrical switching circuits.
- ▶ The Python type name `bool` reflects this logical tradition.



Boolean Logic in the CPU

- ▶ Digital processors implement logical operations using electronic circuits corresponding to operations such as AND, OR, and NOT.
- ▶ Intermediate computational results may be stored temporarily in CPU registers.
- ▶ At the hardware level, logical states are represented using binary electrical states.

Conceptual connection

Boolean expression $\rightarrow$ logical operations $\rightarrow$ binary result $\rightarrow$ true/false interpretation

Image Placeholder

`cpu-boolean-logic-gates.png`

Short-Circuit Evaluation

- ▶ Python's `and` and `or` operators use short-circuit evaluation.
- ▶ For `and`, evaluation can stop once an operand determines that the complete expression cannot be true.
- ▶ For `or`, evaluation can stop once an operand determines that the complete expression is already true.

Examples from the prior deck

```
3 < 2 and 5 < 6 and 7 == 7 and 10 >= 10 and 12 != 13
```

```
3 > 5 or 6 < 4 or 7 == 7 or 10 < 9 or 12 != 12
```

Placement

Defer this behavior until students need to reason about evaluation side effects, function calls, or control-flow conditions.

De Morgan's Laws

First law

not (A and B)

is logically equivalent to

not A or not B

Second law

not (A or B)

is logically equivalent to

not A and not B

- ▶ De Morgan's Laws transform negated compound Boolean expressions into logically equivalent forms.
- ▶ They are useful for simplifying and reasoning about more complex logical conditions.